

TEC | Tecnológico
de Costa Rica

Curso Seguridad del Software

por Adriana Rojas



CÁMARAS
ENCENDIDAS
POR FAVOR

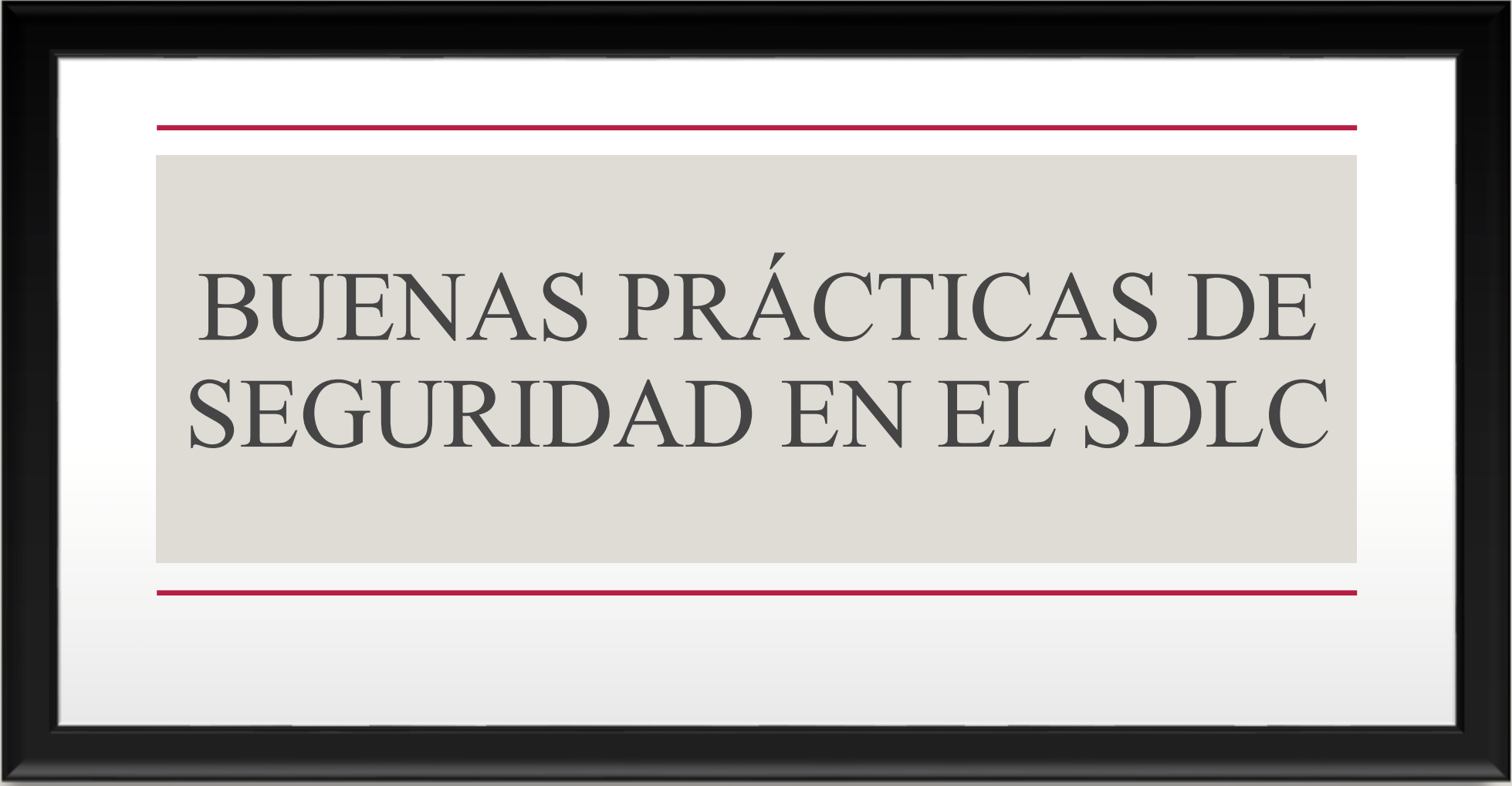


DECLARACIÓN DE USO ÉTICO DE INTELIGENCIA ARTIFICIAL Y FUENTES EXTERNAS

- Se **autoriza y fomenta** el uso de herramientas de Inteligencia Artificial Generativa (como Gemini, ChatGPT, Claude, etc.), así **como del material complementario** proporcionado en clase, como apoyo para la fase de investigación, estructuración y generación de ideas. Sin embargo, el uso de estas herramientas está sujeto a las siguientes condiciones obligatorias:
 - **Responsabilidad y Autoría:** La IA puede generar alucinaciones (datos falsos o códigos erróneos). Es responsabilidad absoluta del estudiante verificar la veracidad técnica de las respuestas.
 - **Prohibición del "Copy-Paste":** No se aceptarán respuestas que sean una transcripción literal de la IA.
 - **Transparencia (Anexo de Prompts):** En caso de haber utilizado IA, se debe incluir un **Anexo** al final del documento que contenga:
 - La herramienta utilizada (ej. Gemini 1.5 Pro).
 - Copia íntegra de los **prompts** (instrucciones) enviados a la herramienta. Y los cambios hechos a dichos prompts para obtener una respuesta satisfactoria.

TEMA 2: CICLO DE VIDA DE DESARROLLO DE SOFTWARE





BUENAS PRÁCTICAS DE SEGURIDAD EN EL SDLC

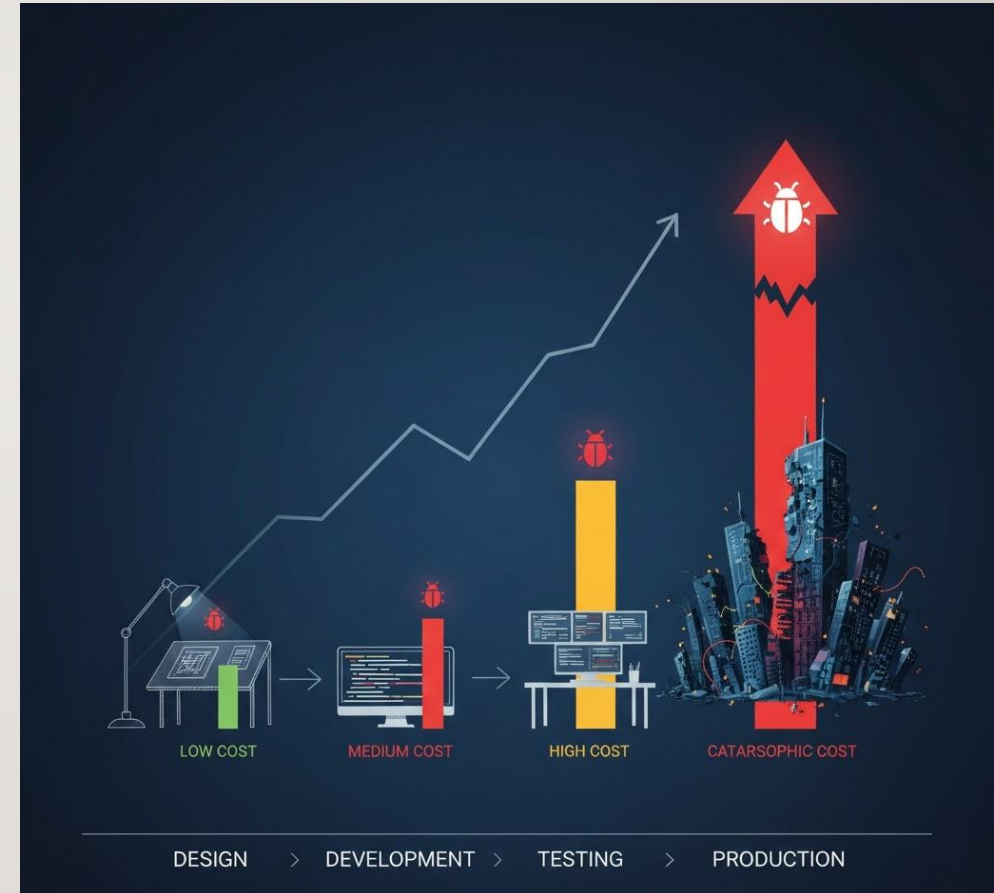
EL PROBLEMA

EL COSTO DE CORREGIR UN ERROR DE
SEGURIDAD EN PRODUCCIÓN VS. EN
DISEÑO (LEY DE COSTO EXPONENCIAL).



LA LEY DEL COSTO EXPONENCIAL

- Es uno de los principios más sagrados (y temidos) en la ingeniería de software. Básicamente, dicta que **cuanto más tarde se encuentre un error o vulnerabilidad, más caro será repararlo.**
- No es un aumento lineal (de 1 en 1), sino **exponencial**: el costo se dispara violentamente conforme el software avanza en su ciclo de vida.



PRINCIPIOS FUNDAMENTALES

CONFIDENCIALIDAD,
INTEGRIDAD Y
DISPONIBILIDAD (LA TRÍADA CIA).

TRIADA DE LA CIBERSEGURIDAD



TRIADA DE LA CIBERSEGURIDAD Y SEGURIDAD DEL SOFTWARE

Cómo confidencialidad, integridad y disponibilidad se traducen en prácticas concretas de desarrollo, pruebas y operación de software

Confidencialidad: control de acceso, cifrado en tránsito y en reposo, gestión de secretos



Integridad: firmas, hashing, control de versiones y revisión de código automatizada



Disponibilidad: diseño resiliente, tolerancia a fallos y planificación de capacidad



DevSecOps: integrar seguridad en CI/CD para cumplir la triada desde el ciclo de vida



Pruebas: pruebas de penetración, análisis SAST/DAST y verificación de integridad



Monitoreo y respuesta: detección de anomalías y recuperación rápida para mantener disponibilidad



Políticas y cumplimiento: auditorías, gestión de parches y control de configuración



FASE DE REQUERIMIENTOS Y DISEÑO

MODELADO DE AMENAZAS (THREAT MODELING)

Es un proceso **estructurado** para:

- Identificar **qué puede salir mal**
- Analizar **cómo podría ser atacado un sistema**
- Definir **controles de seguridad** desde el diseño

No busca encontrar bugs de código, sino **riesgos de diseño**.
¿Cuándo se hace?

- En la **fase de diseño del software**
- Antes del Desarrollo
- Cada vez que hay cambios importantes en arquitectura, datos o funcionalidades

PASOS DEL MODELADO DE AMENAZAS (FASE DE DISEÑO) - PASO 1

1. Definir el sistema a analizar

- Se describe **qué es el sistema y qué hace**, sin entrar en código.
- Incluye:
 - Objetivo del sistema
 - Tipo de aplicación (web, móvil, API)
 - Usuarios (clientes, admins, terceros)
 - Datos que procesa (personales, sensibles, financieros)
- *Ejemplo FlashPresto:*
Aplicación de pedidos y pagos que procesa datos personales, historial de consumo y pagos digitales.

PASOS DEL MODELADO DE AMENAZAS (FASE DE DISEÑO) - PASO 2

2. Identificar los activos críticos

- Los **activos** son aquello que necesita protección.
- Ejemplos:
 - Datos personales (nombre, correo, teléfono)
 - Datos sensibles (pagos, ubicación)
 - Credenciales
 - Reputación del negocio
 - Disponibilidad del servicio
- *Pregunta clave:*
¿Qué pasaría si este activo se pierde, se filtra o se altera?

PASOS DEL MODELADO DE AMENAZAS (FASE DE DISEÑO) - PASO 3

3. Dibujar el Diagrama de Flujo de Datos (DFD)

- Se crea un **diagrama simple**, no técnico, que muestre:
 - Usuarios
 - Aplicación
 - Servidores
 - Bases de datos
 - Servicios externos
 - Flujos de datos entre ellos
- Aquí se marcan **límites de confianza (trust boundaries)**, por ejemplo:
 - Entre usuario y servidor
 - Entre la app y servicios externos
- Este paso es crucial porque **la mayoría de amenazas aparecen en los flujos de datos**, no en los componentes aislados.

PASOS DEL MODELADO DE AMENAZAS (FASE DE DISEÑO) - PASO 4

4. Identificar amenazas potenciales

- Se analizan los flujos y componentes preguntando:
 - “¿Qué podría salir mal aquí?”
- Se consideran amenazas como:
 - Acceso no autorizado
 - Robo de credenciales
 - Manipulación de datos
 - Exposición de datos personales
 - Abuso de funcionalidades legítimas

PASOS DEL MODELADO DE AMENAZAS (FASE DE DISEÑO) - PASO 5

5. Evaluar impacto y probabilidad

- Cada amenaza se evalúa según:
 - **Impacto** (alto, medio, bajo)
 - **Probabilidad** (alta, media, baja)
- Esto permite **priorizar**, porque no todo se puede mitigar al mismo nivel.
- *Ejemplo:*
 - Fuga de datos personales es un impacto alto
 - Error de visualización es un impacto bajo

PASOS DEL MODELADO DE AMENAZAS (FASE DE DISEÑO) - PASO 6

6. Definir mitigaciones desde el diseño

- Aquí está el verdadero valor del Threat Modeling.
- Para cada amenaza se definen controles como:
 - Minimización de datos
 - Cifrado
 - Autenticación fuerte
 - Control de accesos
 - Registro y monitoreo
 - Consentimiento explícito
 - Separación de privilegios
- *Importante:*
Las mitigaciones se **diseñan**, no se “parchan” después.

PASOS DEL MODELADO DE AMENAZAS (FASE DE DISEÑO) - PASO 7

7. Documentar y validar

- El resultado final es un documento simple que incluya:
 - Diagrama
 - Lista de amenazas
 - Riesgos priorizados
 - Decisiones de diseño
- Este documento sirve para:
 - Desarrolladores
 - Equipos legales
 - Auditorías
 - Evidencia de *Security & Privacy by Design*

PRIVACIDAD DESDE EL DISEÑO

- Threat Modeling:
 - Apoya **Privacidad desde el Diseño** (Ley 8968)
 - Reduce riesgos de **delitos informáticos** (Ley 9048)
 - Demuestra diligencia y responsabilidad del desarrollador

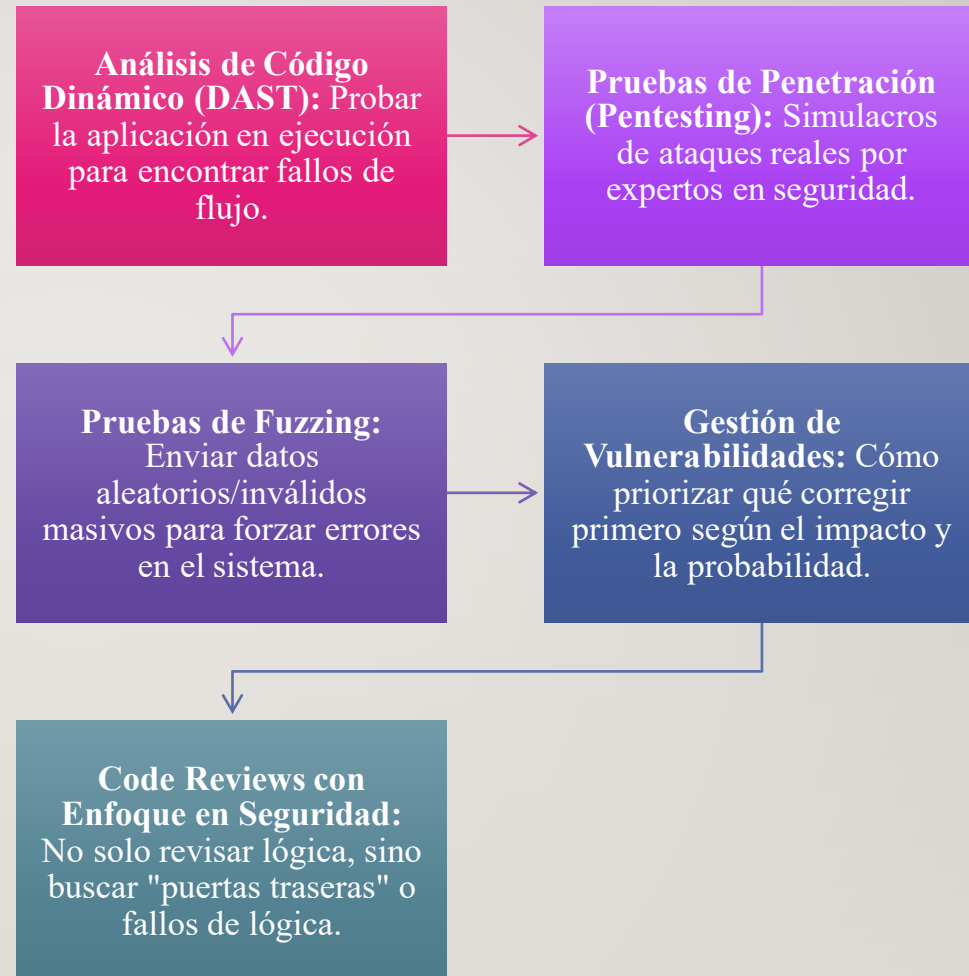
CLASE 4

VAMOS A TRABAJAR A PARTIR DE ESTE PUNTO DURANTE ESTA CLASE

FASE DE FASE DE DESARROLLO / CODIFICACIÓN

FASE DE PRUEBAS Y VERIFICACIÓN

TIPOS DE PRUEBAS



ESTÁNDARES DE DESARROLLO DE SOFTWARE SEGURO

- Existen organizaciones que han creado "bibliotecas" de estándares y marcos de trabajo (frameworks) para que las empresas no tengan que adivinar cómo proteger su código.

EXISTEN ORGANIZACIONES QUE HAN CREADO "BIBLIOTECAS" DE ESTÁNDARES Y MARCOS DE TRABAJO (FRAMEWORKS) PARA QUE LAS EMPRESAS NO TENGAN QUE ADIVINAR CÓMO PROTEGER SU CÓDIGO



OWASP

El estándar global de seguridad en
aplicaciones Web

La fundación sin fines de lucro que
redefine la seguridad del software



Comunidad

Miles de profesionales a nivel mundial colaboran de forma abierta para compartir conocimientos, resolver retos y generar guías de seguridad imparciales.



Herramientas

Ofrece tecnologías de código abierto esenciales (como OWASP ZAP) para auditar, probar y defender aplicaciones sin barreras económicas.



Estándares

Documentos técnicos reconocidos globalmente como marcos de cumplimiento, guiando el ciclo de vida del desarrollo seguro en la industria.

PILARES DE OWASP

EL FAMOSO OWASP TOP 10

- **Llamado el Estándar de Oro**
- El OWASP Top 10 es un documento de consenso que representa los diez riesgos de seguridad más críticos para las aplicaciones web.
- Es reconocido por organizaciones de todo el mundo como el punto de partida definitivo para implementar defensas robustas y adoptar prácticas de codificación segura.

NAVEGUEMOS EN OWASP

- <https://owasp.org/>
- <https://owasp.org/www-project-top-ten/>

SEGURIDAD EN LA CADENA DE SUMINISTRO (SCA)

- La **Seguridad en la Cadena de Suministro de Software (SCA)** se enfoca en identificar y reducir riesgos derivados del uso de:

- Librerías
- Frameworks
- Dependencias de terceros
- Componentes open source

Idea clave

- Tu aplicación es tan segura como **la dependencia más débil** que utilices.

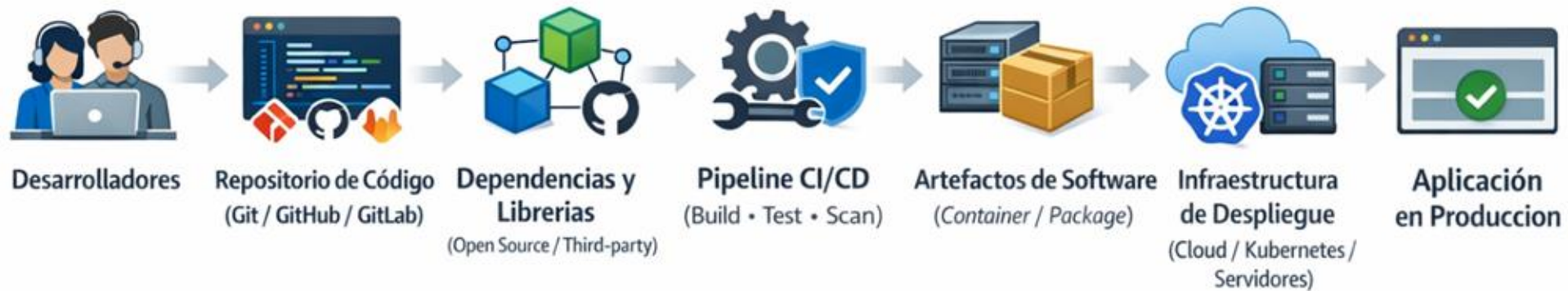
LA CADENA DE SUMINISTROS DE SOFTWARE

A03:2025 Software Supply Chain Failures

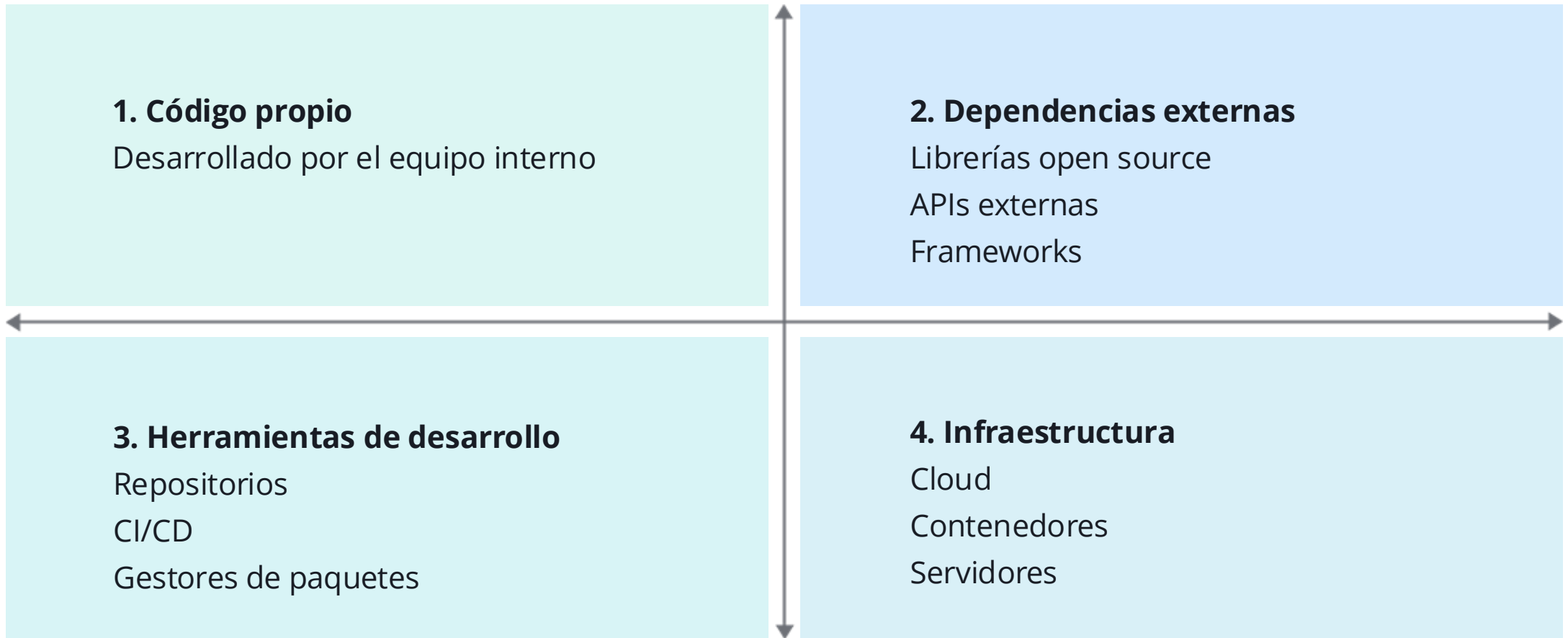
“Hoy el software no se escribe completamente... se ensambla. Y cada componente que ensamblamos se convierte en parte de nuestra superficie de ataque.”

Cadena de Suministro de Software

El conjunto de procesos y componentes que intervienen en el desarrollo y entrega de una aplicación.



LA CADENA DE SUMINISTROS DE SOFTWARE



CLASE 5

VAMOS A TRABAJAR A PARTIR DE ESTE PUNTO DURANTE ESTA CLASE

NIST (NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY)

El estándar de la innovación

Liderazgo en Ciencia, Estándares y Tecnología





Ciencia de Medición

Desarrollo de metrología avanzada para garantizar precisión en la industria global.



Normas Técnicas

Creación de estándares que facilitan la innovación y la interoperabilidad comercial.



Competitividad

Impulso a la industria para mantener el liderazgo tecnológico y económico.

MISIÓN Y PILARES ESTRÁTEGICOS

HISTORIA Y NATURALEZA

- **Agencia Federal**
- Fundado en 1901 como parte del Departamento de Comercio de los Estados Unidos.
- Es una agencia **no reguladora**, lo que significa que sus estándares son adoptados voluntariamente debido a su calidad técnica.
- Sede principal en Gaithersburg, Maryland, con laboratorios de clase mundial.

NIST CSF 2.0

- El estándar de oro para la gestión de riesgos cibernéticos

FUNCIONES DEL FRAMEWORK CSF 2.0



Gobernar

Establecer la estrategia y política de gestión de riesgos.



Identificar

Comprender los activos y riesgos de la organización.



Proteger

Implementar salvaguardas para limitar el impacto.



Detectar

Identificar la ocurrencia de un evento de ciberseguridad.



Responder

Tomar medidas ante un incidente detectado.



Recuperar

Restaurar capacidades y servicios afectados.

EVOLUCIÓN: CSF 1.1 VS CSF 2.0

Característica	NIST CSF 1.1 (2018)	NIST CSF 2.0 (2024)
Alcance	Infraestructuras Críticas	Todas las organizaciones (Universal)
Funciones Core	5 (ID, PR, DE, RS, RC)	6 (Añade GOVERN / Gobernar)
Enfoque	Operativo / Técnico	Estratégico / Riesgo de Negocio
Recursos	Guías estáticas	Herramientas de referencia online

PUBLICACIONES ESPECIALES (SERIE 800)



NIST SP 800-53: Controles de seguridad y privacidad para sistemas de información.



NIST SP 800-171: Protección de información sensible no clasificada en contratistas.



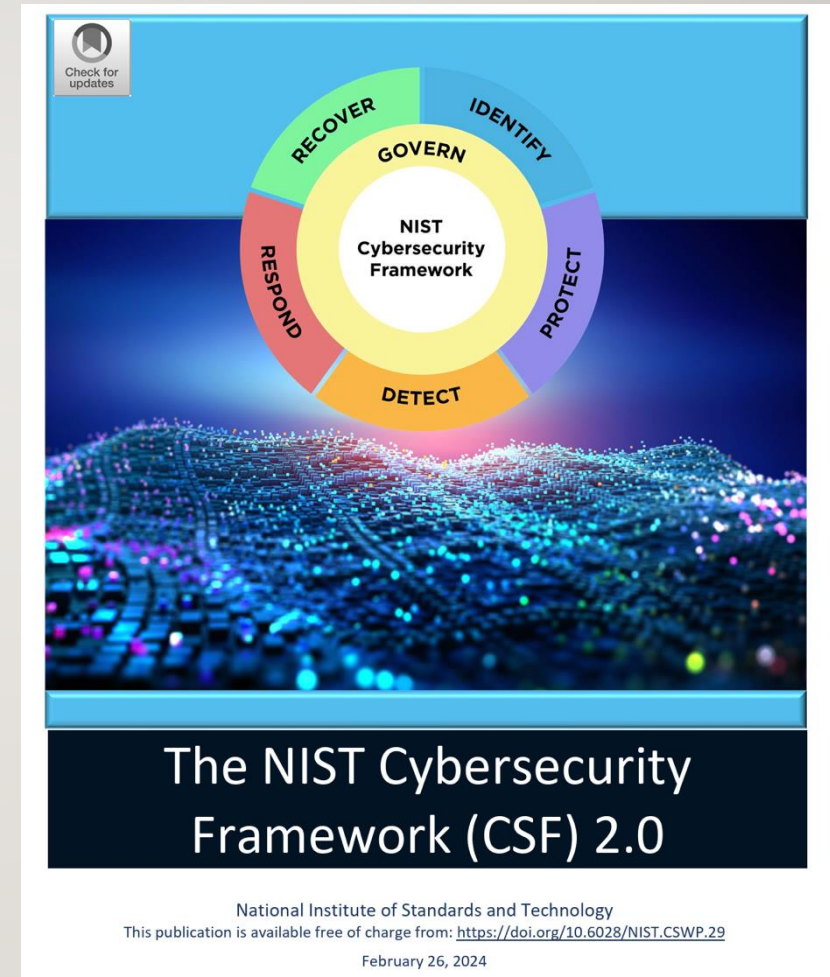
NIST SP 800-30: Guía para realizar evaluaciones de riesgo.



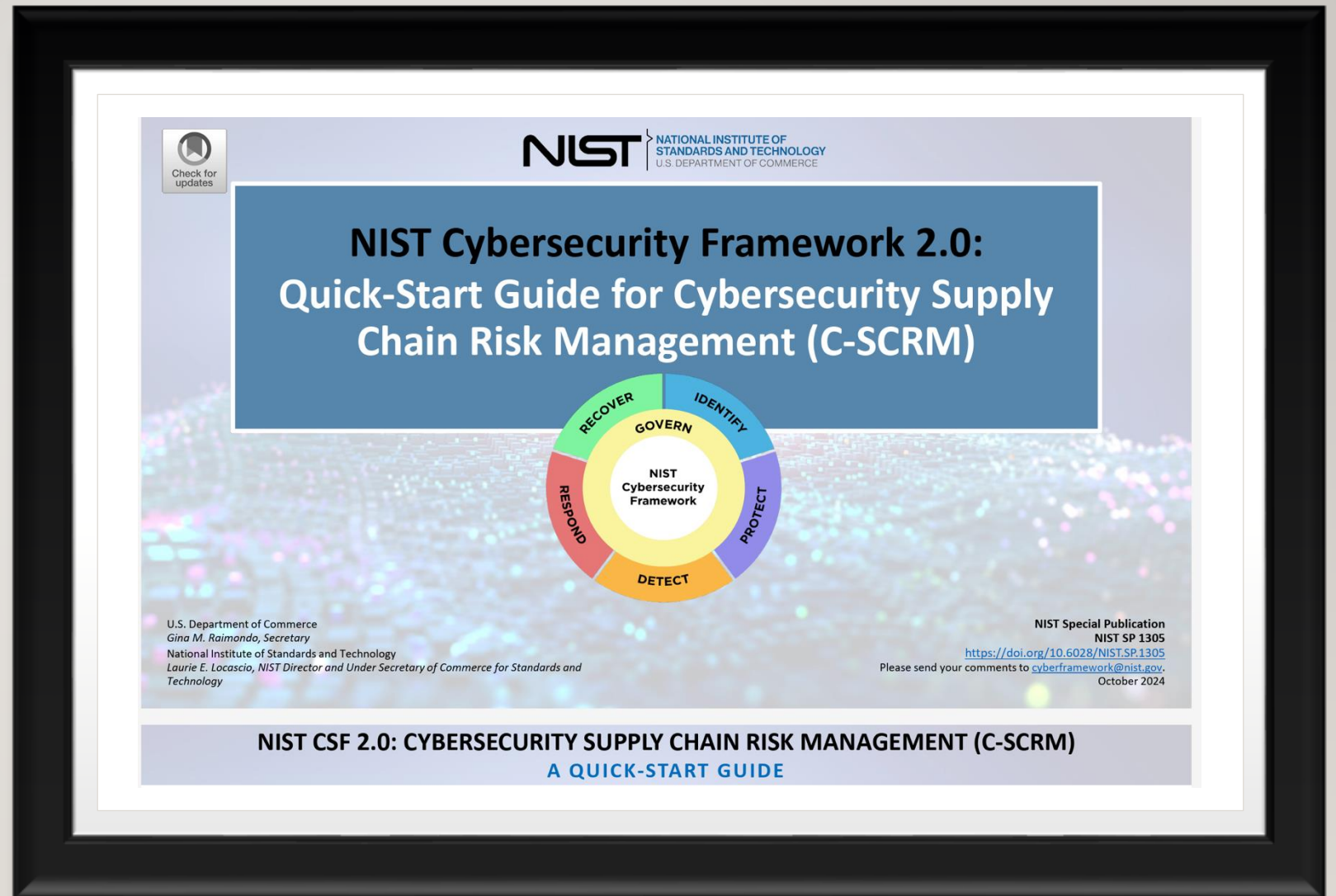
NICE Framework: Definición de roles y competencias en la fuerza laboral de ciberseguridad.

NAVEGUEMOS EN NIST

- <https://www.nist.gov/>
- <https://www.nist.gov/cyberframework/resources-0>
- <https://www.nist.gov/cyberframework>
- <https://www.nist.gov/cyberframework/quick-start-guides#erm>
- <https://csrc.nist.gov/pubs/ir/8355/final>
- <https://www.nist.gov/itl/applied-cybersecurity/nice/nice-framework-resource-center/nice-framework-current-versions>



NIST CSF 2.0: CYBERSECURITY SUPPLY CHAIN RISK MANAGEMENT (C- SCRM)



ISO/IEC (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION)



Gobernanza y Seguridad
en Aplicaciones



ISO/IEC 27001 El pilar de
la gestión de la seguridad
de la información
organizacional

¿QUÉ ES EL SGSI? (ISO/IEC 27001)

Es la norma internacional para implementar un Sistema de Gestión de la Seguridad de la Información (SGSI).

Enfoque Global

Protege la confidencialidad, integridad y disponibilidad de los datos en toda la empresa (procesos, personas y tecnología).

Mejora Continua

Basado en la gestión activa de riesgos y la adopción de controles dinámicos adaptados al negocio.

- ✓ Certificable para empresas.
- ✓ Cumplimiento regulatorio legal.
- ✓ Mitigación de riesgos de negocio.
- ✓ Genera confianza con los clientes.

ISO/IEC 27034

- Seguridad robusta integrada directamente en el ciclo de vida del software

Norma enfocada específicamente en la Seguridad de las Aplicaciones (Application Security).

- 🛡️ Proporciona una guía integral para diseñar, crear, seleccionar y desplegar aplicaciones seguras.
- </> No se limita a revisar el código final; introduce la seguridad desde la fase de requerimientos arquitectónicos.

Concepto Clave: ASC

Introduce los **Controles de Seguridad de la Aplicación (Application Security Controls)** como medidas específicas para mitigar riesgos puntuales del software.

Orientado a Procesos

Ayuda a demostrar de forma auditable que la aplicación mantiene un nivel de riesgo aceptable en todas sus versiones.

ISO/IEC 27001 VS. ISO/IEC 27034

Criterio	ISO/IEC 27001 (SGSI)	ISO/IEC 27034 (Seguridad en Aplicaciones)
Enfoque Principal	Toda la organización (Políticas, TI, RRHH, físico).	Específico para el Ciclo de Vida del Software (SDLC).
Objetivo de Seguridad	Gobernanza global de los activos de información.	Asegurar que el software resista ataques de forma nativa.
Usuarios Clave	Oficiales de Seguridad (CISO), Auditores, Directivos.	Desarrolladores, Arquitectos de Software, DevOps, QA.
Relación entre ambas	Define el marco macro y los objetivos de seguridad del negocio.	Ejecuta y cumple los objetivos técnicos dentro del código y la app.

SANS INSTITUTE

- Líder Mundial en Formación y Certificación de Ciberseguridad

PILARES FUNDAMENTALES DE SANS



Formación

Cursos inmersivos de 5 a 6 días en múltiples dominios: defensa, ataque, forense y gestión.



GIAC

Certificaciones que validan habilidades prácticas, no solo conocimientos teóricos.



Investigación

Proyectos de código abierto, carteles de referencia y el famoso Internet Storm Center.

RECURSOS PARA LA COMUNIDAD

Internet Storm Center
(ISC): Monitorización global de amenazas en tiempo real.

Reading Room: Más de 3,000 artículos técnicos gratuitos escritos por expertos.

SANS
Posters: Hojas de referencia visuales para comandos de Linux, Windows y más.

NewsBites: Boletín informativo semanal con las noticias más críticas de seguridad.

VAMOS A INVESTIGAR

- Ingresar a <https://www.sans.org/>
- Hacerse una cuenta

CLASE 6

VAMOS A TRABAJAR A PARTIR DE ESTE PUNTO DURANTE ESTA CLASE

Análisis de riesgo orientado a la
arquitectura y marcos de trabajo para el
análisis de riesgo

-
- El Análisis de Riesgos Orientado a la Arquitectura (también conocido como *Architectural Risk Analysis* o ARA) es abordarlo no como un simple checklist de vulnerabilidades, sino como una evaluación estructural. No estamos buscando fallos de código (como un punto y coma mal puesto o una falta de sanitización básica), sino fallos de diseño estructural que comprometan todo el sistema.
 - Vamos a trabajar explicando: el concepto analógico, el framework de referencia y el proceso práctico.

DIFERENCIA ENTRE UN DEFECTO DE IMPLEMENTACIÓN (BUG) Y UN DEFECTO DE DISEÑO (VULNERABILIDAD ARQUITECTÓNICA)

- Si estuviésemos construyendo un edificio, un **bug de código** es usar un ladrillo quebrado o dejar una ventana mal sellada. Es un problema local y fácil de cambiar.
- Un **riesgo de arquitectura** es haber diseñado las columnas principales demasiado delgadas para soportar el peso del techo, o haber puesto la bóveda del banco justo al lado de la ventana de la calle. Por más que uses los mejores ladrillos del mundo, el edificio colapsará porque el diseño estructural está mal.



EL FRAMEWORK DE REFERENCIA: NIST SP 800-30 + SSDF

- **NIST SP 800-30 (Guía para Evaluaciones de Riesgo):** Nos da la base metodológica de cómo identificar amenazas, vulnerabilidades, impacto y probabilidad.
- **NIST SSDF (Secure Software Development Framework - SP 800-218):** En su pilar de "*Diseñar Software Seguro*" (PO - Protect the Software), exige explícitamente evaluar la arquitectura y **realizar modelado de amenazas** antes de escribir la primera línea de código.



EL PROCESO PRÁCTICO DE LA EVALUACIÓN EN 4 PASOS



PASO A: DESCOMPOSICIÓN DEL SISTEMA (VER EL PLANO)

- No se puede evaluar lo que no se conoce. El primer paso consiste en dibujar la arquitectura usando diagramas técnicos que muestren:
- **Componentes:** Servicios, bases de datos, APIs de terceros, microservicios.
- **Flujos de Datos:** Cómo viaja la información entre esos componentes.
- **Límites de Confianza (Trust Boundaries):** Marcar claramente dónde cambia el nivel de privilegio (por ejemplo, la línea divisoria entre el navegador del usuario de internet y nuestra red interna).

PASO B: IDENTIFICACIÓN DE AMENAZAS (¿QUÉ PODRÍA SALIR MAL?)

- Aquí aplicamos metodologías complementarias como **STRIDE** (de Microsoft) sobre el diagrama diseñado. Analizamos cada componente y flujo buscando:
- *Suplantación (Spoofing)*: ¿Puede un componente falso hacerse pasar por nuestra base de datos?
- *Alteración (Tampering)*: ¿Alguien puede modificar los datos en tránsito entre el API Gateway y el microservicio?
- *Elevación de Privilegios*: ¿Un usuario común puede saltarse la lógica y hablarle directo al backend de administración?

METODOLOGÍAS COMPLEMENTARIAS **STRIDE** (DE MICROSOFT)

- Ver ppt separada

PASO C:
DETERMINACIÓN
DEL RIESGO
(CÁLCULO SEGÚN
NIST SP 800-30)

- Para cada amenaza identificada, se cruzan dos variables para obtener el nivel de riesgo:
- **Probabilidad (Likelihood):** Qué tan viable o fácil es que el escenario ocurra considerando los controles existentes.
- **Impacto (Impact):** Qué tan grave sería el daño para el negocio (pérdida financiera, reputación, fuga de datos).

$$\text{Riesgo} = \text{Probabilidad} \times \text{Impacto}$$

PASO D: MITIGACIÓN E INGENIERÍA DE REQUERIMIENTOS DE SEGURIDAD

- El análisis de riesgo no es solo para documentar problemas, es para resolverlos. El resultado final del ARA es la generación de **requerimientos de seguridad para los desarrolladores**.
- *Si el riesgo arquitectónico es:* "Fuga de credenciales porque el microservicio A se autentica con el microservicio B en texto plano".
- *El requisito de diseño será:* "Implementar mTLS (Mutual TLS) obligatorio para toda la comunicación Este-Oeste dentro del clúster".

-
- El Análisis de Riesgo Orientado a la Arquitectura consiste en **evaluar el plano de la casa antes de comprar los materiales**. Nos permite identificar fallas estructurales destructivas cuando aún son baratas y fáciles de corregir en el papel, garantizando un software resiliente por diseño.

CÓMO CALCULAR EL RIESGO EXACTO USANDO LA FÓRMULA DE LA GUÍA NIST SP 800-30

Paso A: Determinar el Índice de Probabilidad

Para obtener el valor de probabilidad, debes cruzar la **frecuencia o viabilidad del ataque** con la robustez de los controles existentes.

Paso B: Determinar el Índice de Impacto

El impacto mide la gravedad de las consecuencias sobre el negocio o los activos de información si la amenaza llega a consumarse de manera exitosa. Se evalúa considerando la pérdida de **Confidencialidad, Integridad y Disponibilidad (CIA)**:

Paso C: Aplicar la Matriz y Obtener el Valor Final

Una vez determinados ambos índices individuales, se multiplican para obtener la puntuación final del riesgo

Casos de Abuso y Requerimientos de Seguridad en Software

FUNDAMENTOS DE LOS CASOS DE ABUSO

Caso de Uso (Funcional)

Describe interacciones positivas entre el usuario y el sistema para alcanzar una meta de negocio.

Enfoque: ¿Qué debe hacer el software?

Caso de Abuso (Seguridad)

Describe interacciones maliciosas donde un actor intenta causar daño real al sistema o a sus datos.

Enfoque: ¿Qué NO debe permitir el software?

EL "MISUSER": EL ACTOR ANTAGONISTA

- A diferencia del usuario legítimo, el actor de abuso (Misuser) tiene metas que entran en conflicto con la seguridad del sistema.
- Motivación: Ganancia financiera, espionaje o sabotaje.
- Capacidad: Desde usuarios curiosos hasta atacantes organizados.
- Recursos: Herramientas automatizadas y técnicas de ingeniería social.

ANATOMÍA DE UN CASO DE ABUSO



Meta del Abuso

El objetivo final del atacante (ej: exfiltrar datos de tarjetas de crédito).



Amenaza

El método o vector utilizado (ej: Inyección SQL en el formulario de login).



Impacto

La consecuencia negativa para la organización (ej: multa regulatoria y daño de marca).

DE LA AMENAZA AL REQUERIMIENTO

Atributo (CIA)	Caso de Abuso	Requerimiento de Seguridad
Confidencialidad	Sniffing de credenciales	Cifrado TLS 1.3 en todas las conexiones
Integridad	Tampering de precios	Validación de firmas en el lado del servidor
Disponibilidad	Ataque DoS por recursos	Implementación de Rate Limiting y Cuotas
Trazabilidad	Borrado de logs	Inmutabilidad de bitácoras de auditoría

SEGURIDAD POR DISEÑO

- Integrar requerimientos de seguridad en la arquitectura previene fallos estructurales que son imposibles de corregir solo con código.
 - Defensa en Profundidad
 - Principio de Mínimo Privilegio
 - Reducción de Superficie de Ataque